Table 2. Largest size of E1485 (weak central groupoids) models exhausted by each configuration in under 300s. The `skolems_last` parameter turns out to be irrelevant for this problem.

Configuration	(2,4,Y/N)	(0,4,Y/N)	(2,3,Y/N)	(2,2,Y/N)	(2,1,Y/N)
Size	14 (in 127 s)	14 (in 166 s)	13 (in 290 s)	9 (in 33 s)	7 (in 8 s)

In addition, while in the ETP we typically searched for a single model to contradict a given implication, in some cases we sought all models of a certain size for some given theory, either to better understand a collection of base examples in order to extend them, or just to improve our understanding. Accordingly, it should be noted that a configuration that is fast for finding one model of a given size may be slower than others for exhausting the whole size. A notable example of this is (2,2,Y) for models of E677 of size 9, which finds a single model in 160s, but is unable to exhaust the whole size after 20h (see Table 3).

When undertaking a long-running model search at a large size, it is advisable to determine in advance the potentially optimal configuration(s) by examining smaller sizes. Throughout the ETP, we have employed the following procedure. If time permits, exhaust the previous size with all configurations to make a choice. If that process is prohibitively time-consuming, then set the target number of desired configurations (e.g., for parallel runs), initialize the pool of configurations with all possibilities, and iterate:

(1) Pick the next available smallest size.
(2) Run all configurations on that size under a reasonable time limit and collect their running times, whether for finding a single model, a predetermined number of models, or for exhausting the size.
(3) Based on the smallest time found, determine a statistically significant time threshold.
(4) Remove from the pool those configurations whose time differs from the best one by more than the threshold (if any). End the loop once the number of configurations contained in the pool is the desired one.

Be aware that this algorithm assumes that once a configuration outperforms another, this advantage carries over to larger sizes. This is not always the case (see Table 4 for an example). Additionally, the algorithm may be further refined by considering combinations of the configurations with different subsets of formulas.

7.3.7. *Search of compatible properties.* When in search of a model, either theoretically or using a model builder, it is useful to be able to identify additional properties to impose on the model, strong enough to simplify the search, yet weak enough to guarantee compatibility with the original axioms. In particular, when seeking a countermodel to implication $\mathrm{E} \models \mathrm{E}'$, we should avoid properties that, together with E, would imply E'. This search can be greatly aided by an ATP: if we have an educated guess that some property P is compatible with the problem $\mathrm{E} \not\models \mathrm{E}'$, we can run the ATP on $\mathrm{E} \wedge P$ under various strategies to attempt a proof of E'. If, after allowing ample time, no proof is found, this provides heuristic evidence that a countermodel satisfying P exists. Note that even if no proof actually exists, it may well be that there are infinite countermodels but not finite ones.

TABLE 3. Time to find one model of E677 of size 9 and to exhaust the same size for different configurations. Some finite-setting formulas were included to speed up the search (see Section 7.3.8). The configurations not listed here were unable to find a model in 4000 s.

Configuration	One model	Exhaust size
(2,1,N)	16 s	190 s
(2,1,Y)	16 s	210 s
(0,1,Y)	20 s	149 s
(0,1,N)	32 s	169 s
(2,2,Y)	154 s	>72 000 s
(2,2,N)	180 s	>43 000 s
(2,3,N)	557 s	877 s
(2,3,Y)	587 s	941 s
(2,4,Y)	596 s	761 s
(2,4,N)	620 s	730 s
(0,3,N)	3746 s	4151 s
(0,3,Y)	3777 s	4254 s

TABLE 4. Comparison of the two best configurations for finding certain special E677 models, both for exhausting each size and for finding a single model, with best times in boldface. Note that initially (2,4,N) seems the best configuration, but in the long run, (2,3,Y) outperforms it. The comparison is particularly misleading at size 16 when searching for a single model. All times are in seconds.

	Exhaustion		Finding one model	
Size	(2,3,Y)	(2,4,N)	(2,3,Y)	(2,4,N)
9	0.14	**0.04**	0.02	**0.0**
10	0.32	**0.12**	—	—
11	0.82	**0.37**	0.15	**0.01**
12	2.17	**1.2**	—	—
13	6.18	**4.35**	2.74	**0.91**
14	15.47	**14.39**	—	—
15	**38.82**	50.87	—	—
16	**120**	175	11.95	**0.02**
17	**306**	660	—	—
18	**883**	2382	—	—
19	**2300**	>3600	**846**	2082

In the ETP, we successfully applied this approach to several of the outstanding implications. Most notably, we were able to construct an infinite model of E1323 $\not\models$ E2744 after heuristically verifying compatibility[19] with a unit element and with closure of the operation over the set of square elements.

[19]Specifically, for each property, we ran *Prover9* for 20 minutes with the default configuration and *Vampire* for 999s in CASC mode.

7.3.8. *Finite setting.* There are situations in which one wants to work in the finite setting, for example, when proving a finite implication. Indeed, model builders typically search for finite models; therefore, when using them, we can usually assume we are operating in the finite setting. In this context, injective (resp. surjective) maps are bijective, bijections are periodic maps, and so on. In addition, any law of the form $x = f(y) \diamond g(x, y)$ for some maps f, g has left multiplication map $L_{f(y)}$ surjective, hence bijective. Thus by applying the substitution $x \mapsto f(y) \diamond x$ and simplifying $f(y)$ on the left, the law finitely implies $x = g(f(y) \diamond x, y)$.

By this and similar approaches we typically find some multiplication or related map (squaring, etc.) to be injective and surjective, properties that we may add to our initial formulas. In hindsight, adding injectivity to a finite model builder usually increases performance, whereas surjectivity (and other existentially quantified formulas) tend to diminish it. Moreover, *Prover9-Mace4* performs better with operations and equalities than with non-equational formulas. For this reason it is advantageous to convert an injectivity condition from an implication into a new operation. For example, injectivity of the left multiplication map is captured by $x \backslash (x * y) = y$ (where $*$ stands for $\diamond$); this substitution typically yields a threefold speedup. In contrast, this conversion appears to slow down *Vampire* slightly.

Example 7.1. As a case study, let us examine the model search for E1518 $\not\models$ E47 using *Mace4* (with $*$ standing for $\diamond$). It is now known that the smallest models have size 15. Initially, this search exceeded our ATPs expertise, hence we ended up constructing a theoretical model of size 232. If we restrict to the original formulas E1518 and ¬E47, the optimal configuration is (2,3,N), taking 2.5 minutes to exhaust size 10 and already exceeding 7 hours to exhaust size 11. Putting $S(x) := x \diamond x$, since E1518 is $x = (y \diamond y) \diamond (x \diamond (y \diamond x))$ we observe that $L_{S(y)}$ is surjective, hence injective. Accordingly we add `(y*y)*x = (y*y)*z -> x = z` to *Mace4* and then size 11 is exhausted in less than 5 minutes. In addition, from the implication graph we observe that E1518 implies E359, which states $S(x) \diamond x = S(x)$; when added as `(x*x)*x = x*x`, *Mace4* exhausts size 11 in under 1s, with the optimal configuration shifting to (2,1,N). Size 12 is exhausted in 18s. We also note that if $S(x) = S(y)$ then $S(x) \diamond x = S(x) = S(y) = S(y) \diamond y$, yielding $x = y$ by injectivity of $L_{S(x)}$. Therefore S is injective, we can add `x*x = y*y -> x = y`, and exhaust size 12 in 13.5s. Also, in E1518 we can substitute $x \mapsto S(y) \diamond x$ and cancel $S(y)$ on the left to get E320858, with which we exhaust size 12 in under 2s and size 13 in under 1 minute. Moreover, as S is surjective every element is a square, so the injectivity of $L_{S(x)}$ implies that of L_x, and we can add `x*y = x*z -> y = z` to the search. But this actually worsens the performance! After some experimentation, we observe that we should either add the injectivity of $L_{S(x)}$ or that of L_x, but not both. It turns out that for size 13, injectivity of $L_{S(x)}$ performs slightly better, so we retain it. Additionally, the implication formula for this injectivity can be replaced with a new operation \ by writing `x\((x*x)*y) = y`. This further reduces the exhausting time for size 13 to 33s. Now size 14 is exhausted in 30 minutes, and size 15 is well within reach.

7.3.9. *Discriminators for Isofilter. Mace4*'s output may be redundant, in that many of the models found in a search may (and often will) be isomorphic to each other. To solve this problem, *Mace4*'s output can be fed to another tool called *Isofilter*, which returns a representative model of each isomorphism class. *Isofilter* compares permutations of the models, restricted according to some discriminators: by default the only one used is the frequency with which